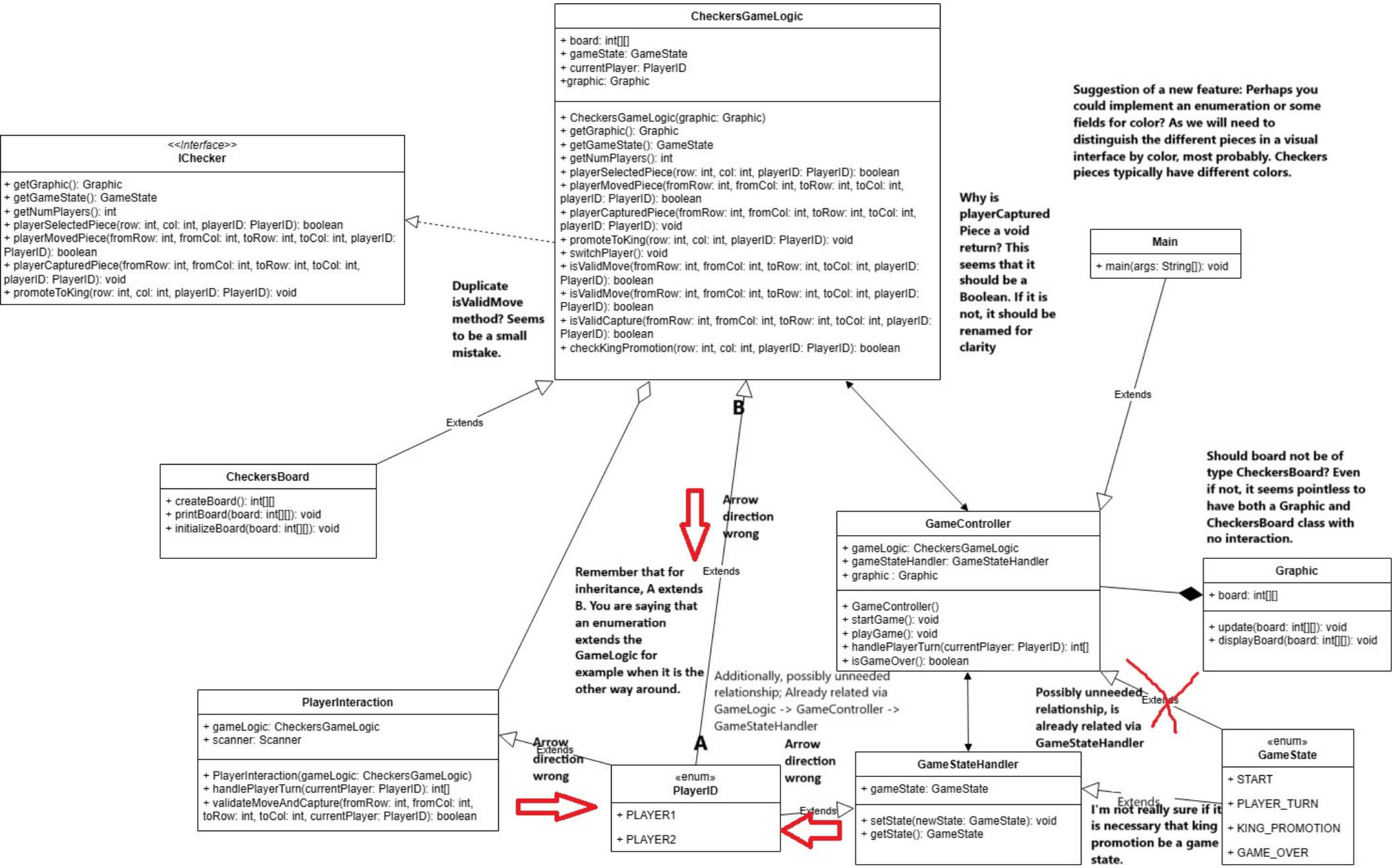

CheckersGameLogic
+ board: int[][]
+ gameState: GameState
+ currentPlayer: PlayerID
+graphic: Graphic
+ CheckersGameLogic(graphic: Graphic)
+ getGraphic(): Graphic
+ getGameState(): GameState
+ getNumPlayers(): int
+ playerSelectedPiece(row: int, col: int, playerID: PlayerID): boolean
+ playerMovedPiece(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): boolean
+ playerCapturedPiece(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): void
+ promoteToKing(row: int, col: int, playerID: PlayerID): void
+ switchPlayer(): void
+ isValidMove(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): boolean
+ isValidMove(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): boolean
+ isValidCapture(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): boolean
+ checkKingPromotion(row: int, col: int, playerID: PlayerID): boolean
<<Interface>>
IChecker
+ getGraphic(): Graphic
+ getGameState(): GameState
+ getNumPlayers(): int
+ playerSelectedPiece(row: int, col: int, playerID: PlayerID): boolean
+ playerMovedPiece(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): boolean
+ playerCapturedPiece(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): void
+ promoteToKing(row: int, col: int, playerID: PlayerID): void
Duplicate isValidMove method? Seems to be a small mistake.
Why is playerCaptured Piece a void return? This seems that it should be a Boolean. If it is not, it should be renamed for clarity
Suggestion of a new feature: Perhaps you could implement an enumeration or some fields for color? As we will need to distinguish the different pieces in a visual interface by color, most probably. Checkers pieces typically have different colors.
Main
+ main(args: String[]): void
Extends
CheckersBoard
+ createBoard(): int[][]
+ printBoard(board: int[][]): void
+ initializeBoard(board: int[][]): void
B
Arrow direction wrong
Remember that for inheritance, A extends B. You are saying that an enumeration extends the GameLogic for example when it is the other way around.
Extends
GameController
+ gameLogic: CheckersGameLogic
+ gameStateHandler: GameStateHandler
+ graphic : Graphic
+ GameController()
+ startGame(): void
+ playGame(): void
+ handlePlayerTurn(currentPlayer: PlayerID): int[]
+ isGameOver(): boolean
Should board not be of type CheckersBoard? Even if not, it seems pointless to have both a Graphic and CheckersBoard class with no interaction.
Graphic
+ board: int[][]
+ update(board: int[][]): void
+ displayBoard(board: int[][]): void
Additionally, possibly unneeded relationship; Already related via GameLogic -> GameController -> GameStateHandler
Possibly unneeded relationship, is already related via GameStateHandler
Extends
PlayerInteraction
+ gameLogic: CheckersGameLogic
+ scanner: Scanner
+ PlayerInteraction(gameLogic: CheckersGameLogic)
+ handlePlayerTurn(currentPlayer: PlayerID): int[]
+ validateMoveAndCapture(fromRow: int, fromCol: int, toRow: int, toCol: int, currentPlayer: PlayerID): boolean
Arrow direction wrong
Extends
A
«enum»
PlayerID
+ PLAYER1
+ PLAYER2
Arrow direction wrong
Extends
GameStateHandler
+ gameState: GameState
+ setState(newState: GameState): void
+ getState(): GameState
Extends
«enum»
GameState
+ START
+ PLAYER_TURN
+ KING_PROMOTION
+ GAME_OVER
I'm not really sure if it is necessary that king promotion be a game state.